homework-4

2026-10-5

Dylan Smith Homework 4

Q1. Data Splitting and Reproducibility

```
set.seed(2026)
train.set <- sample(1:47, 35)
test.set <- setdiff(1:47, train.set)
```

Q2. Feature Selection via ANOVA and Feature Normalization

(a) One-Way ANOVA for Each Gene Using the Training Set

```
expression.matrix <- t(as.matrix(chromoX[, 3:49]))
storage.mode(expression.matrix) <- "numeric"
colnames(expression.matrix) <- make.unique(as.character(chromoX$IDENTIFIER))

training.groups <- factor(BLClabels[train.set])
anova.p.values <- apply(expression.matrix[train.set, , drop = FALSE], 2, function(x) {
  anova.table <- summary(aov(x ~ training.groups))[[1]]
  anova.table[1, "Pr(>F)"]
})

anova.results <- data.frame(
  Gene = names(anova.p.values),
  P_value = as.numeric(anova.p.values),
  row.names = NULL
)
```

(b) Select the Five Genes with the Smallest p-Values

```
selected.genes <- names(sort(anova.p.values))[1:5]
selected.genes
```

```
## [1] "AR"      "AR.1"    "CXorf61" "XIST.4"  "XIST.1"
```

(c) Form the 47 by 5 Reduced Feature Matrix

```r
selected.features <- expression.matrix[, selected.genes, drop = FALSE]
dim(selected.features)
```

```
## [1] 47  5
```

(d) Min-Max Normalize the Selected Features Across All Samples

```r
feature.mins <- apply(selected.features, 2, min)
feature.maxs <- apply(selected.features, 2, max)
normalized.features <- sweep(
  sweep(selected.features, 2, feature.mins, "-"),
  2, feature.maxs - feature.mins, "/"
)
dim(normalized.features)
```

```
## [1] 47  5
```

Q3. Model Training

```r
model.training.data <- as.data.frame(
  normalized.features[train.set, , drop = FALSE]
)
model.training.data$Class <- factor(
  BLClabels[train.set],
  levels = c(1, 2, 3),
  labels = c("BLC", "non-BLC", "Normal")
)

# Multinomial Logistic Regression
multinomial.model <- nnet::multinom(
  Class ~ ., data = model.training.data, trace = FALSE
)

# LDA
lda.model <- MASS::lda(Class ~ ., data = model.training.data)

# Naive Bayes
naive.bayes.model <- e1071::naiveBayes(
  Class ~ ., data = model.training.data
)

# K-NN
knn.training.features <- model.training.data[
  , setdiff(names(model.training.data), "Class"), drop = FALSE
]
knn.training.classes <- model.training.data$Class
knn.k <- c(4, 8, 12)

summary(multinomial.model)
```

```
## Call:
## nnet::multinom(formula = Class ~ ., data = model.training.data,
##     trace = FALSE)
##
## Coefficients:
##         (Intercept)        AR      AR.1   CXorf61    XIST.4    XIST.1
## non-BLC   -22.52561 -29.69911  49.42399 -51.53463 194.60058 -168.0563
## Normal     80.28028 -76.43627 -26.55066  74.54174 -47.57317  -80.4097
##
## Std. Errors:
##         (Intercept)        AR       AR.1    CXorf61   XIST.4    XIST.1
## non-BLC    25.04485  54.63289   70.02456   58.81211 221.4862  190.8623
## Normal    525.90277 428.75228 1185.45474 1214.39548 656.2023 1127.9595
##
## Residual Deviance: 4.745212
## AIC: 28.74521
```

```
lda.model
```

```
## Call:
## lda(Class ~ ., data = model.training.data)
##
## Prior probabilities of groups:
##       BLC   non-BLC    Normal
## 0.2571429 0.2857143 0.4571429
##
## Group means:
##                 AR      AR.1    CXorf61    XIST.4    XIST.1
## BLC     0.64615385 0.6260174 0.02821797 0.6464877 0.7252403
## non-BLC 0.73682606 0.7237121 0.02946121 0.7627070 0.7586637
## Normal  0.08084719 0.2430487 0.62429379 0.3020417 0.2998680
##
## Coefficients of linear discriminants:
##               LD1        LD2
## AR      -4.032594   3.915301
## AR.1     1.188186  -6.404703
## CXorf61  1.952118  -1.030272
## XIST.4  -1.372851 -17.765790
## XIST.1  -2.021388  15.975468
##
## Proportion of trace:
##    LD1    LD2
## 0.9218 0.0782
```

```
naive.bayes.model
```

```
##
## Naive Bayes Classifier for Discrete Predictors
##
## Call:
## naiveBayes.default(x = X, y = Y, laplace = laplace)
##
## A-priori probabilities:
```

```
## Y
##       BLC   non-BLC    Normal
## 0.2571429 0.2857143 0.4571429
##
## Conditional probabilities:
##          AR
## Y               [,1]      [,2]
##   BLC     0.64615385 0.2846552
##   non-BLC 0.73682606 0.1832391
##   Normal  0.08084719 0.1256948
##
##          AR.1
## Y              [,1]      [,2]
##   BLC     0.6260174 0.2098125
##   non-BLC 0.7237121 0.1423559
##   Normal  0.2430487 0.1365608
##
##          CXorf61
## Y               [,1]       [,2]
##   BLC     0.02821797 0.02257331
##   non-BLC 0.02946121 0.02911847
##   Normal  0.62429379 0.35178112
##
##          XIST.4
## Y              [,1]      [,2]
##   BLC     0.6464877 0.1535301
##   non-BLC 0.7627070 0.1222739
##   Normal  0.3020417 0.2007819
##
##          XIST.1
## Y              [,1]      [,2]
##   BLC     0.7252403 0.1877569
##   non-BLC 0.7586637 0.1147294
##   Normal  0.2998680 0.2299144
```

```
knn.k
```

```
## [1]  4  8 12
```

Q4. Decision Boundary Equations

(a) Multinomial Logistic Regression Coefficients and Decision Boundaries

```
multinomial.coefficients <- coef(multinomial.model)
feature.names <- setdiff(names(model.training.data), "Class")

format.multinomial.linear <- function(beta) {
  slope.terms <- sprintf("%+.6f * %s", beta[feature.names], feature.names)
  paste(sprintf("%.6f", beta["(Intercept)"]),
        paste(slope.terms, collapse = " "))
}
```

```
multinomial.boundaries <- c(
  "BLC vs non-BLC" = paste0(
    "0 = ", format.multinomial.linear(multinomial.coefficients["non-BLC", ])
  ),
  "BLC vs Normal" = paste0(
    "0 = ", format.multinomial.linear(multinomial.coefficients["Normal", ])
  ),
  "non-BLC vs Normal" = paste0(
    "0 = ", format.multinomial.linear(
      multinomial.coefficients["non-BLC", ] -
        multinomial.coefficients["Normal", ]
    )
  )
)

multinomial.coefficients
```

```
##         (Intercept)        AR      AR.1   CXorf61    XIST.4    XIST.1
## non-BLC   -22.52561 -29.69911  49.42399 -51.53463 194.60058 -168.0563
## Normal     80.28028 -76.43627 -26.55066  74.54174 -47.57317  -80.4097
```

```
multinomial.boundaries
```

```
##
##  "0 = -22.525607 -29.699113 * AR +49.423985 * AR.1 -51.534625 * CXorf61 +194.600575 * XIST.4 -168.05
##
##     "0 = 80.280279 -76.436267 * AR -26.550662 * AR.1 +74.541744 * CXorf61 -47.573174 * XIST.4 -80.40
##                                                                                                  non-
## "0 = -102.805886 +46.737154 * AR +75.974647 * AR.1 -126.076370 * CXorf61 +242.173749 * XIST.4 -87.64
```

(b) LDA Projection Directions and Decision Boundary Lines

```
lda.projection.directions <- lda.model$scaling
lda.projection.equations <- vapply(
  seq_len(ncol(lda.projection.directions)),
  function(j) {
    slopes <- lda.projection.directions[feature.names, j]
    paste0(colnames(lda.projection.directions)[j], " = ",
           paste(sprintf("%+.6f * %s", slopes, feature.names), collapse = " "))
  },
  character(1)
)

lda.training.scores <- predict(
  lda.model, newdata = knn.training.features
)$x
lda.class.levels <- levels(model.training.data$Class)
lda.class.means <- lapply(lda.class.levels, function(class.name) {
  colMeans(lda.training.scores[model.training.data$Class == class.name, , drop = FALSE])
})
```

```r
names(lda.class.means) <- lda.class.levels

lda.within.residuals <- do.call(rbind, lapply(lda.class.levels, function(class.name) {
  class.scores <- lda.training.scores[model.training.data$Class == class.name, , drop = FALSE]
  sweep(class.scores, 2, lda.class.means[[class.name]], "-")
}))
lda.within.covariance <- crossprod(lda.within.residuals) /
  (nrow(lda.training.scores) - length(lda.class.levels))
lda.class.priors <- prop.table(table(model.training.data$Class))

lda.class.pairs <- combn(lda.class.levels, 2, simplify = FALSE)
lda.boundary.table <- do.call(rbind, lapply(lda.class.pairs, function(class.pair) {
  mean.1 <- lda.class.means[[class.pair[1]]]
  mean.2 <- lda.class.means[[class.pair[2]]]
  direction <- solve(lda.within.covariance, mean.1 - mean.2)
  intercept <- -0.5 * drop(crossprod(mean.1 + mean.2, direction)) +
    log(lda.class.priors[class.pair[1]] / lda.class.priors[class.pair[2]])

  data.frame(
    Class_1 = class.pair[1], Class_2 = class.pair[2],
    LD1 = direction[1], LD2 = direction[2], Intercept = intercept,
    Equation = sprintf("%.6f * LD1 %+.6f * LD2 %+.6f = 0",
                       direction[1], direction[2], intercept)
  )
}))

lda.projection.directions
```

```
##               LD1        LD2
## AR      -4.032594   3.915301
## AR.1     1.188186  -6.404703
## CXorf61  1.952118  -1.030272
## XIST.4  -1.372851 -17.765790
## XIST.1  -2.021388  15.975468
```

```r
lda.projection.equations
```

```
## [1] "LD1 = -4.032594 * AR +1.188186 * AR.1 +1.952118 * CXorf61 -1.372851 * XIST.4 -2.021388 * XIST.1
## [2] "LD2 = +3.915301 * AR -6.404703 * AR.1 -1.030272 * CXorf61 -17.765790 * XIST.4 +15.975468 * XIST
```

```r
lda.boundary.table
```

```
##      Class_1 Class_2        LD1        LD2  Intercept
## LD1      BLC non-BLC  0.4742513  1.8027498  0.7100226
## LD11     BLC  Normal -4.3209394  1.0508295  0.3163622
## LD12 non-BLC  Normal -4.7951907 -0.7519202 -0.3936604
##                                        Equation
## LD1   0.474251 * LD1 +1.802750 * LD2 +0.710023 = 0
## LD11 -4.320939 * LD1 +1.050830 * LD2 +0.316362 = 0
## LD12 -4.795191 * LD1 -0.751920 * LD2 -0.393660 = 0
```

Q5. Visualization of Decision Boundaries

Choose Two Features and Create a Prediction Grid

```
plot.features <- colnames(knn.training.features)[1:2]
grid.resolution <- 140
feature1.values <- seq(0, 1, length.out = grid.resolution)
feature2.values <- seq(0, 1, length.out = grid.resolution)
prediction.grid <- expand.grid(
  feature1.values,
  feature2.values
)
names(prediction.grid) <- plot.features

training.feature.means <- colMeans(knn.training.features)
grid.data <- as.data.frame(lapply(training.feature.means, function(value) {
  rep(value, nrow(prediction.grid))
}))
names(grid.data) <- names(training.feature.means)
grid.data[[plot.features[1]]] <- prediction.grid[[plot.features[1]]]
grid.data[[plot.features[2]]] <- prediction.grid[[plot.features[2]]]

class.colors <- c(
  "BLC" =  "lightblue",
  "non-BLC" = "white",
  "Normal" = "grey"
)
training.classes <- knn.training.classes
```

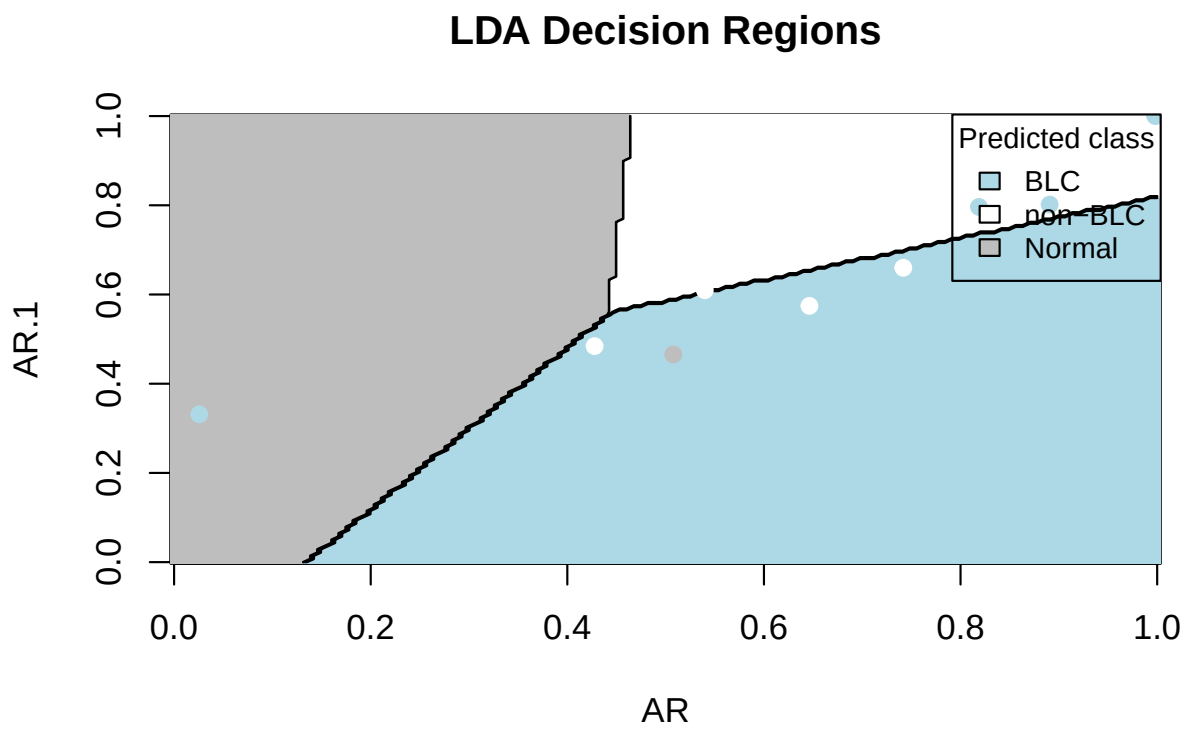
Plot Decision Regions and Training Points

```
plot.class.regions <- function(predicted.classes, plot.title) {
  region.matrix <- matrix(
    as.integer(predicted.classes),
    nrow = length(feature1.values),
    ncol = length(feature2.values)
  )

  image(
    x = feature1.values, y = feature2.values, z = region.matrix,
    col = unname(class.colors), breaks = seq(0.5, 3.5, by = 1),
    xlab = plot.features[1], ylab = plot.features[2],
    main = plot.title, useRaster = TRUE
  )
  contour(
    x = feature1.values, y = feature2.values, z = region.matrix,
    levels = c(1.5, 2, 2.5), add = TRUE, drawlabels = FALSE,
    col = "black", lwd = 1.2
  )
  points(
    knn.training.features[[plot.features[1]]],
```

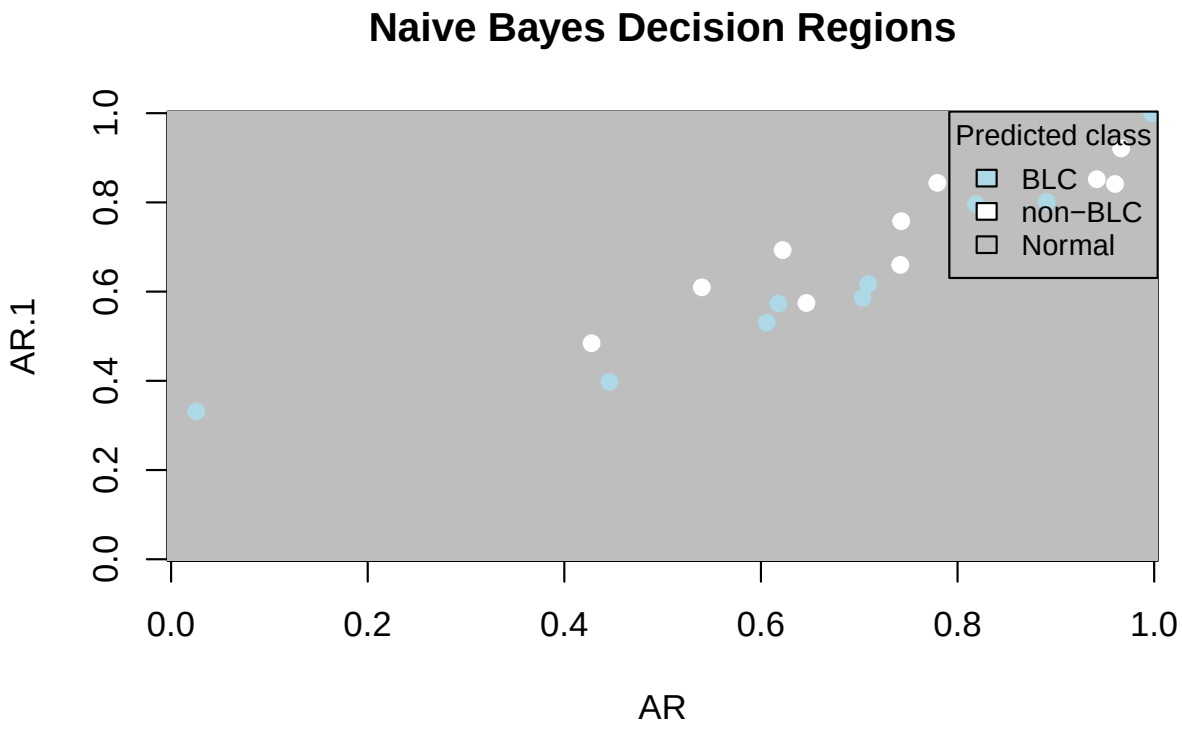
```
    knn.training.features[[plot.features[2]]],
    pch = 19, col = class.colors[as.character(training.classes)]
  )
  legend(
    "topright", legend = names(class.colors),
    fill = unname(class.colors), title = "Predicted class", cex = 0.8
  )
}

lda.grid.classes <- predict(lda.model, newdata = grid.data)$class
plot.class.regions(lda.grid.classes, "LDA Decision Regions")
```




```
naive.bayes.grid.classes <- predict(
  naive.bayes.model, newdata = grid.data, type = "class"
)
plot.class.regions(naive.bayes.grid.classes, "Naive Bayes Decision Regions")
```

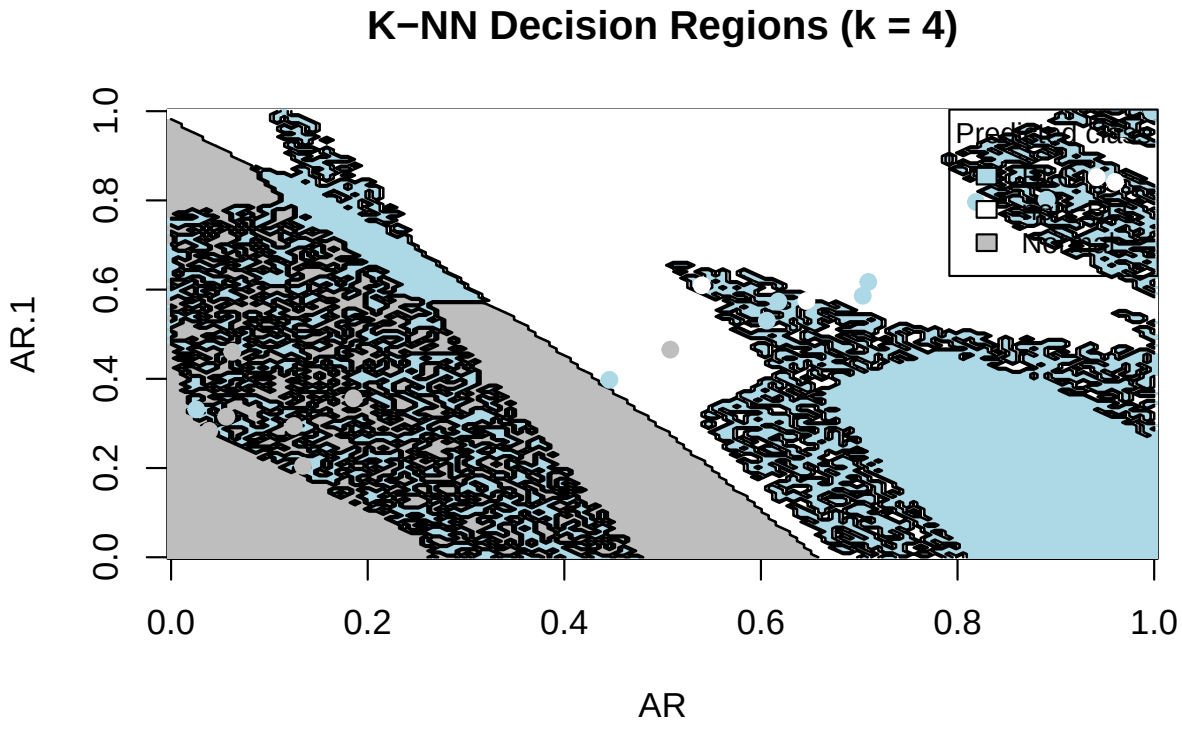
```
## Warning in contour.default(x = feature1.values, y = feature2.values, z =
## region.matrix, : all z values are equal
```

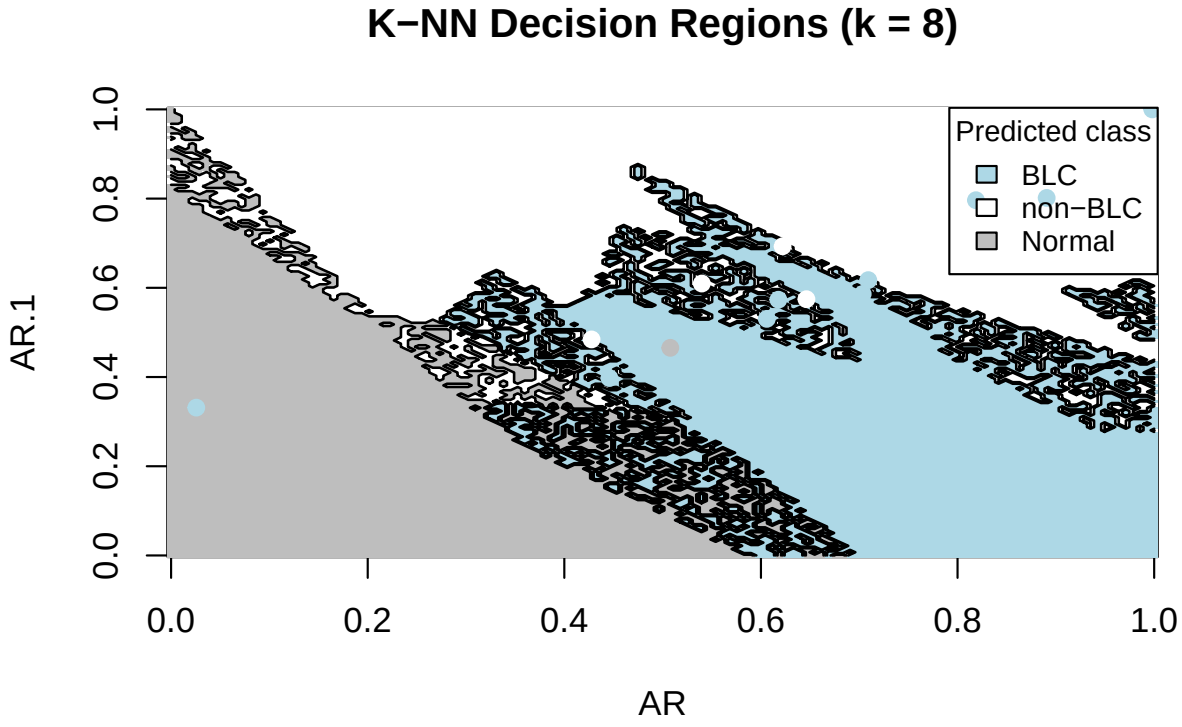


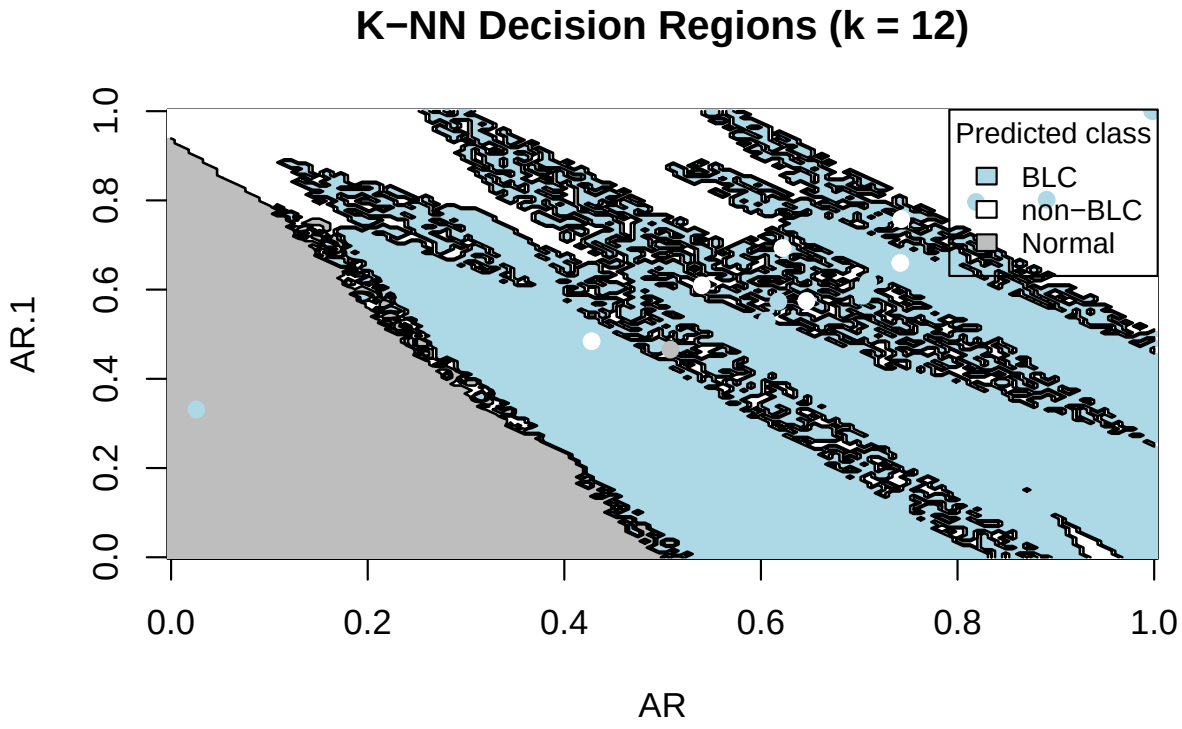
K-NN Decision Regions for k = 4, 8, 12, and 5

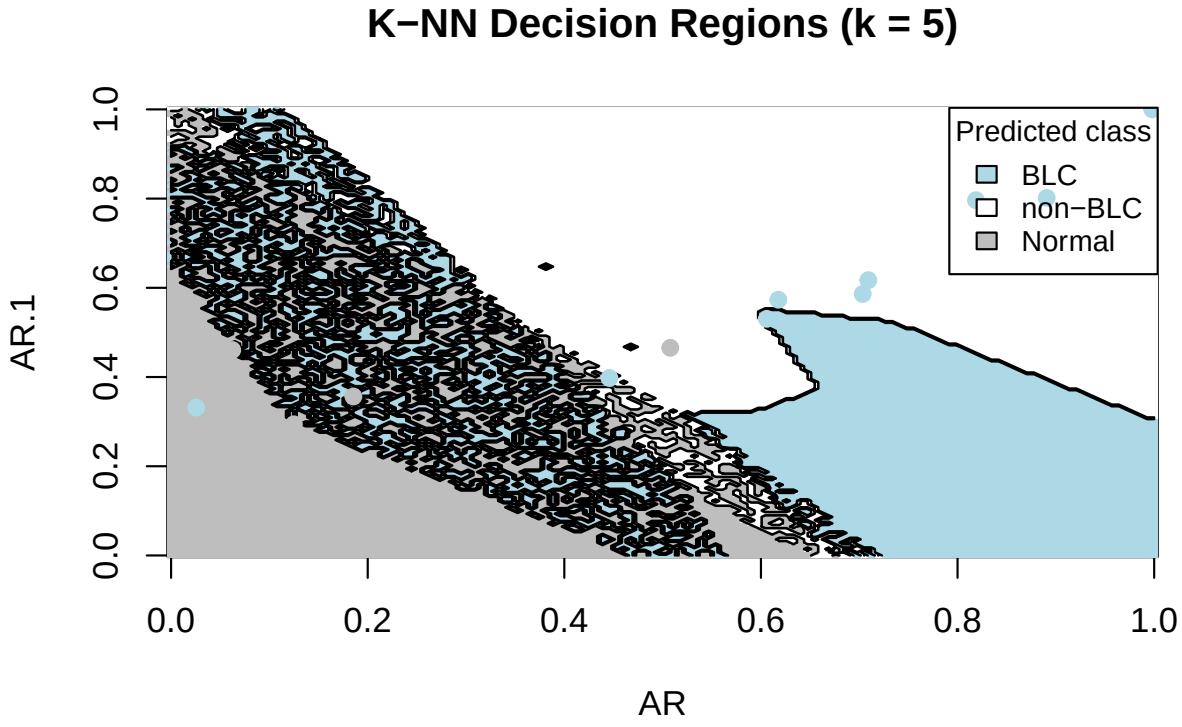
```
knn.plot.values <- c(4, 8, 12, 5)
knn.grid.classes <- setNames(
  lapply(knn.plot.values, function(k) {
    class::knn(
      train = knn.training.features,
      test = grid.data,
      cl = knn.training.classes,
      k = k
    )
  }),
  paste0("k = ", knn.plot.values)
)

for (model.name in names(knn.grid.classes)) {
  plot.class.regions(
    knn.grid.classes[[model.name]],
    paste("K-NN Decision Regions (", model.name, ")", sep = "")
  )
}
```


K−NN Decision Regions (k = 4)
AR.1
AR
0.0
0.2
0.4
0.6
0.8
1.0


K−NN Decision Regions (k = 8)
Predicted class
BLC
non−BLC
Normal
AR.1
AR
0.0
0.2
0.4
0.6
0.8
1.0


K−NN Decision Regions (k = 12)
Predicted class
BLC
non−BLC
Normal
AR.1
AR
0.0
0.2
0.4
0.6
0.8
1.0


K−NN Decision Regions (k = 5)
Predicted class
BLC
non−BLC
Normal
AR.1
AR
0.0
0.2
0.4
0.6
0.8
1.0

Q6. Test Set Evaluation & Performance Comparison

(a) Holdout Predictions and Confusion Matrices

```
test.features <- as.data.frame(
  normalized.features[test.set, , drop = FALSE]
)
class.levels <- levels(model.training.data$Class)
test.classes <- factor(
  BLClabels[test.set], levels = c(1, 2, 3),
  labels = c("BLC", "non-BLC", "Normal")
)

test.predictions <- list(
  "Multinomial Logistic Regression" = predict(
    multinomial.model, newdata = test.features, type = "class"
  ),
  "LDA" = predict(lda.model, newdata = test.features)$class,
  "Naive Bayes" = predict(
    naive.bayes.model, newdata = test.features, type = "class"
  ),
  "K-NN (k = 4)" = class::knn(
    train = knn.training.features, test = test.features,
    cl = knn.training.classes, k = 4
  ),
  "K-NN (k = 8)" = class::knn(
    train = knn.training.features, test = test.features,
    cl = knn.training.classes, k = 8
  ),
  "K-NN (k = 12)" = class::knn(
    train = knn.training.features, test = test.features,
    cl = knn.training.classes, k = 12
  )
)
test.predictions <- lapply(test.predictions, function(x) {
  factor(as.character(x), levels = class.levels)
})

multiclass.confusion.matrices <- lapply(test.predictions, function(predicted) {
  table(
    Actual = factor(test.classes, levels = class.levels),
    Predicted = factor(predicted, levels = class.levels)
  )
})
for (method in names(multiclass.confusion.matrices)) {
  cat("\n", method, "\n", sep = "")
  print(multiclass.confusion.matrices[[method]])
}
```

```
##
## Multinomial Logistic Regression
##          Predicted
## Actual    BLC non-BLC Normal
```

```
##   BLC       2       2      0
##   non-BLC   1       3      0
##   Normal    0       2      2
##
## LDA
##          Predicted
## Actual    BLC non-BLC Normal
##   BLC       2       2      0
##   non-BLC   1       3      0
##   Normal    0       1      3
##
## Naive Bayes
##          Predicted
## Actual    BLC non-BLC Normal
##   BLC       1       3      0
##   non-BLC   1       3      0
##   Normal    1       0      3
##
## K-NN (k = 4)
##          Predicted
## Actual    BLC non-BLC Normal
##   BLC       2       2      0
##   non-BLC   4       0      0
##   Normal    1       1      2
##
## K-NN (k = 8)
##          Predicted
## Actual    BLC non-BLC Normal
##   BLC       1       3      0
##   non-BLC   3       1      0
##   Normal    0       1      3
##
## K-NN (k = 12)
##          Predicted
## Actual    BLC non-BLC Normal
##   BLC       1       3      0
##   non-BLC   1       3      0
##   Normal    1       0      3
```

```r
binary.levels <- c("Non-normal", "Normal")
actual.binary <- factor(
  ifelse(test.classes == "Normal", "Normal", "Non-normal"),
  levels = binary.levels
)
to.binary.class <- function(x) {
  factor(ifelse(x == "Normal", "Normal", "Non-normal"), levels = binary.levels)
}
binary.confusion.matrices <- lapply(test.predictions, function(predicted) {
  table(Actual = actual.binary, Predicted = to.binary.class(predicted))
})
for (method in names(binary.confusion.matrices)) {
  cat("\n", method, " (Normal is positive)\n", sep = "")
  print(binary.confusion.matrices[[method]])
}
```

```
##
## Multinomial Logistic Regression (Normal is positive)
##             Predicted
## Actual       Non-normal Normal
##   Non-normal          8      0
##   Normal              2      2
##
## LDA (Normal is positive)
##             Predicted
## Actual       Non-normal Normal
##   Non-normal          8      0
##   Normal              1      3
##
## Naive Bayes (Normal is positive)
##             Predicted
## Actual       Non-normal Normal
##   Non-normal          8      0
##   Normal              1      3
##
## K-NN (k = 4) (Normal is positive)
##             Predicted
## Actual       Non-normal Normal
##   Non-normal          8      0
##   Normal              2      2
##
## K-NN (k = 8) (Normal is positive)
##             Predicted
## Actual       Non-normal Normal
##   Non-normal          8      0
##   Normal              1      3
##
## K-NN (k = 12) (Normal is positive)
##             Predicted
## Actual       Non-normal Normal
##   Non-normal          8      0
##   Normal              1      3
```

(b) False Positives, False Negatives, Precision, Recall, and F1-Score

```r
performance.results <- do.call(rbind, lapply(names(binary.confusion.matrices), function(method) {
  confusion <- binary.confusion.matrices[[method]]
  true.negative <- confusion["Non-normal", "Non-normal"]
  false.positive <- confusion["Non-normal", "Normal"]
  false.negative <- confusion["Normal", "Non-normal"]
  true.positive <- confusion["Normal", "Normal"]

  precision <- if (true.positive + false.positive == 0) {
    NA_real_
  } else {
    true.positive / (true.positive + false.positive)
  }
  recall <- if (true.positive + false.negative == 0) {
```

```
    NA_real_
  } else {
    true.positive / (true.positive + false.negative)
  }
  f1.score <- if (2 * true.positive + false.positive + false.negative == 0) {
    NA_real_
  } else {
    2 * true.positive /
      (2 * true.positive + false.positive + false.negative)
  }

  data.frame(
    Method = method,
    FP = as.integer(false.positive), FN = as.integer(false.negative),
    Precision = as.numeric(precision), Recall = as.numeric(recall),
    F1 = as.numeric(f1.score),
    row.names = NULL
  )
}))

knitr::kable(
  performance.results, digits = 3,
  caption = "Holdout performance for Normal (positive) vs. Non-normal"
)
```

Table 1: Holdout performance for Normal (positive) vs. Non-normal

Method	FP	FN	Precision	Recall	F1
Multinomial Logistic Regression	0	2	1	0.50	0.667
LDA	0	1	1	0.75	0.857
Naive Bayes	0	1	1	0.75	0.857
K-NN (k = 4)	0	2	1	0.50	0.667
K-NN (k = 8)	0	1	1	0.75	0.857
K-NN (k = 12)	0	1	1	0.75	0.857